Binary search, step by step

Finding a value in a sorted list

Demo talk

Plan

- The problem: is a value in a sorted list?

Plan

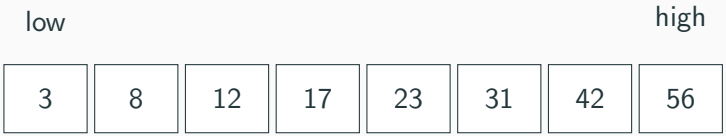
- The problem: is a value in a sorted list?
- The idea: look in the middle, then throw away half

Plan

- The problem: is a value in a sorted list?
- The idea: look in the middle, then throw away half
- The code, and how fast it is

The idea

Looking for 23



Each step halves the part of the list that is left.

Looking for 23

middle: $17 < 23$



Each step halves the part of the list that is left.

Looking for 23

found in 3 steps



Each step halves the part of the list that is left.

In code

A few lines of Python

```
def binary_search(items, target):
    low, high = 0, len(items) - 1
    while low <= high:
        middle = (low + high) // 2
        if items[middle] == target:
            return middle
        if items[middle] < target:
            low = middle + 1 # keep the right half
        else:
            high = middle - 1 # keep the left half
    return -1 # not in the list
```

How fast is it?

Items in the list	Steps at most
8	3
1 000	10
1 000 000	20
1 000 000 000	30

Double the list: just one more step.

Wrap up

Takeaways

- Works only on sorted lists

Takeaways

- Works only on sorted lists
- One comparison removes half of the candidates

Takeaways

- Works only on sorted lists
- One comparison removes half of the candidates
- Try it: `code/binary_search.py`

Questions?